

## 2교시: `compose.yaml` 기본 구조

### 수업 목표

- `services`, `networks`, `volumes` 의 역할을 구분한다.
- `ports`, `environment`, `volumes`, `depends_on`, `healthcheck` 를 읽는다.
- YAML indentation과 variable interpolation 오류를 사전에 찾는다.

### 50분 흐름

| 시간     | 활동                    | 비중     | 산출                    |
|--------|-----------------------|--------|-----------------------|
| 0-8분   | Compose file 전체 구조 보기 | 설명 15% | 구조 note               |
| 8-22분  | web service 읽기        | 설명 25% | port/volume map       |
| 22-36분 | db service 읽기         | 설명 30% | env/health/volume map |
| 36-45분 | config로 해석 결과 확인      | 실행 20% | normalized config     |
| 45-50분 | 오류 유형 정리              | 설명 10% | troubleshooting note  |

Visual 1: Compose file map



이 visual은 Compose file의 top-level 항목을 보여준다. 볼 지점은 service 내부 설정과 top-level network/volume 정의가 분리된다는 점이다.

### 핵심 설명

`compose.yaml` 의 중심은 `services` 다. `service`는 실행할 container의 의도된 역할이다. 예를 들어 `web` 은 `nginx` image를 사용하고 `host port`를 `publish`한다. `db` 는 `PostgreSQL` image를 사용하고 `environment`와 `named volume`을 가진다.

`networks` 와 `volumes` 는 `service` 바깥에서 `project resource`로 정의된다. `service`는 필요한 `network`와 `volume`을 참조한다. 이 구조 덕분에 어떤 `service`가 같은 `network`에 붙는지, 어떤 `data`가 `container` 삭제 뒤에도 남는지 읽을 수 있다.

environment에는 설정 이름이 들어간다. Day 4 실습은 .env에서 값을 주입하지만, 공개 repository에는 실제 secret 값을 올리지는 않는다. Compose file에는 어떤 변수명이 필요한지와 누락되면 어떤 메시지를 낼지만 남긴다.

## 실습 파일 읽기

```
sed -n '1,220p' week2/day4/labs/compose-app/compose.yml
cd week2/day4/labs/compose-app
docker compose config
```

## 주요 항목

| 항목          | 의미                          | Day 3 대응         |
|-------------|-----------------------------|------------------|
| image       | 실행할 image                   | docker run IMAGE |
| ports       | host:container port publish | -p 18084:80      |
| environment | runtime config              | -e KEY=value     |
| volumes     | bind mount 또는 named volume  | -v source:dest   |
| networks    | service 간 통신 boundary       | --network        |
| depends_on  | service 시작 관계               | 수동 실행 순서         |
| healthcheck | readiness evidence          | pg_isready       |

## 핵심 유의사항

YAML은 indentation이 구조다. ports가 web 아래에 있어야 하는데 top-level로 올라가면 의미가 완전히 달라진다. 파일을 눈으로만 보지 말고 docker compose config로 해석 결과를 확인한다.

`\${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}` 형식은 값이 없을 때 명시적으로 실패하게 만든다. 조용히 빈 password로 실행되는 것보다 실행 전에 실패하는 편이 안전하다.

## 자주 놓치는 지점

| 놓치는 지점                 | 증상                        | 확인                    |
|------------------------|---------------------------|-----------------------|
| indentation 오류         | services.web 아래 설정이 사라짐   | docker compose config |
| .env 누락                | variable interpolation 실패 | error message         |
| host/container port 혼동 | wrong port 접속             | ports와 compose ps     |
| named volume 의미 누락     | down -v로 data 삭제          | volumes               |
| healthcheck 과신         | 앱 readiness로 오해           | service별 check 분리     |

## 기록 템플릿

```
## Lesson 2 Compose File Reading
- web image:
- web ports:
- web bind mount:
- db image:
- db env names:
```

- db volume:
- healthcheck:
- config result:

## 평가 기준

| 기준      | 2점 evidence                       |
|---------|-----------------------------------|
| 구조 이해   | top-level과 service-level 항목을 구분했다 |
| mapping | Day 3 옵션과 Compose 항목을 연결했다        |
| 검증      | config 결과로 해석 구조를 확인했다            |

## 공식 근거 링크

- Compose file reference: <https://docs.docker.com/compose/compose-file/>
- Compose services reference: <https://docs.docker.com/reference/compose-file/services/>

## 선행 지식과 범위 경계

이 교시는 YAML 문법 수업이 아니다. YAML indentation은 필요하지만, 핵심은 "Compose file을 읽고 실행 계약을 해석하는 능력"이다.

학생은 Day 2에서 Dockerfile의 instruction을 읽었고, Day 3에서 runtime option을 실행했다. 이제 같은 관점으로 compose.yaml 을 읽는다. FROM , COPY , CMD 가 image build 계약이었다면, services , ports , environment , volumes , networks 는 runtime contract다.

## Compose application model

Compose application model은 application을 service들의 묶음으로 본다. service는 한 종류의 container 역할이고, project는 그 service들이 공유하는 이름공간이다.

| 개념         | 학생용 정의        | 실무 해석                                    |
|------------|---------------|------------------------------------------|
| project    | Compose 실행 단위 | resource name prefix와 isolation boundary |
| service    | container 역할  | web, db, cache 같은 책임 단위                  |
| network    | 통신 경계         | service discovery와 접근 범위                 |
| volume     | data 경계       | container lifecycle과 data lifecycle 분리   |
| config/env | 설정 경계         | image와 환경별 값 분리                          |

이 모델을 이해하면 Compose file을 단순 YAML이 아니라 운영 설계 문서로 읽을 수 있다.

## 학술 기준 연결

CS2023의 competency framework에서는 같은 지식을 다른 representation으로 옮기는 능력을 중요하게 본다. 여기서는 command-line option representation을 YAML application model로 변환한다.

Bloom taxonomy로 보면 이 교시는 apply 와 analyze 단계다. 학생은 주어진 Compose file을 실행할 뿐 아니라, 각 줄이 어떤 runtime effect를 가지는지 분석해야 한다.

NIST NICE 관점에서는 secret과 configuration handling이 포함된다. 비밀번호 값을 file에 쓰는 행위와 필요한 변수 이름을 문서화하는 행위는 다르다. 이 구분은 초급 단계부터 습관화해야 한다.

## services.web 읽기

web service를 읽을 때는 다음 순서로 본다.

1. 어떤 image를 실행하는가?
2. 어떤 port가 host에 publish되는가?
3. source file이 image 안에 있는가, host에서 mount되는가?
4. 다른 service에 의존하는가?
5. 어떤 network에 붙는가?

Day 4 실습의 web 은 nginx image를 사용하고, host 18084 를 container 80 에 publish한다. ./html 을 nginx document root에 read-only bind mount한다. 따라서 HTML 파일 변경은 image rebuild 없이 반영될 수 있다.

## services.db 읽기

db service는 web보다 운영 판단이 많다.

| 항목          | 질문                      | Day 4 기준                 |
|-------------|-------------------------|--------------------------|
| image       | 어떤 DB 버전인가              | postgres:16-alpine       |
| environment | 어떤 초기화 값이 필요한가          | DB, user, password       |
| volume      | data directory가 어디에 남는가 | /var/lib/postgresql/data |
| init script | 최초 초기화 시 무엇을 실행하는가      | db/init SQL              |
| healthcheck | readiness를 어떻게 판단하는가    | pg_isready               |
| network     | 누가 DB에 접근하는가            | app-net 내부 service       |

DB service는 data persistence와 secret handling이 같이 등장하므로 실무 위험도가 web보다 높다.

## variable interpolation 기준

Compose는 `\${NAME}` 형식으로 host environment 또는 .env 값을 치환한다. Day 4 실습은 다음 형식을 사용한다.

```
POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
```

이 형식은 값이 없으면 실패한다. 교육적으로 중요한 이유는 "잘못된 기본값으로 조용히 실행되는 것"보다 "실행 전에 명시적으로 실패하는 것"이 안전하기 때문이다.

## indentation 오류 예시

아래는 나쁜 예시다.

```
services:
  web:
    image: nginx:1.27-alpine
ports:
  - "18084:80"
```

ports 가 web service 아래에 있지 않다. 사람이 보기에는 가까워 보여도 Compose가 해석하는 구조는 다르다. 그래서 수업에서는 docker compose config 를 항상 실행한다.

## 실무 설계 판단: container\_name 을 꼭 써야 하는가

초급자는 container 이름을 고정하면 편하다고 느낀다. 그러나 Compose는 service name과 project name으로 resource를 관리한다. container\_name 을 고정하면 같은 repository를 여러 project name으로 동시에 실행하거나 scale하는 상황에서 충돌할 수 있다.

교육용 실습에서는 container 이름이 보이면 이해가 쉬울 수 있다. 하지만 Day 4 기본 예제는 service name을 중심으로 설명한다. 이유는 Week 3 MSA와 Kubernetes Service로 연결하기 좋기 때문이다.

## 오해 점검 문항

| 문항                                   | 기대 답                                           |
|--------------------------------------|------------------------------------------------|
| top-level volumes만 쓰면 DB에 mount되는가   | 아니다. service의 volumes에서도 참조해야 한다               |
| .env는 image 안에 들어가는가                 | 아니다. Compose CLI가 variable interpolation에 사용한다 |
| environment는 secret 저장소인가            | 아니다. 실습용 설정 전달 방식이며 노출 위험을 고려해야 한다             |
| depends_on은 application 연결 성공을 보장하는가 | 아니다. 시작 관계와 readiness는 구분해야 한다                 |

## 실습: config 결과 읽기

학생은 `docker compose config` 출력에서 다음을 찾아 적는다.

web service의 published port:  
db service의 healthcheck command:  
pgdata volume의 실제 Compose resource name:  
app-net network의 실제 Compose resource name:

이 활동은 YAML 원본과 Compose가 normalize한 결과가 어떻게 다른지 보여준다.

## Evidence 수준 구분

| 수준          | 예시                                                   |
|-------------|------------------------------------------------------|
| 약한 evidence | compose.yaml 파일이 있다                                  |
| 중간 evidence | docker compose config를 실행했다                          |
| 강한 evidence | config 출력에서 service, port, volume, network를 해석해 기록했다 |

## 전이 과제

개인 프로젝트에 `compose.yaml` 을 추가한다고 가정하고 다음 표를 채운다.

| 항목                     | 개인 프로젝트 값 |
|------------------------|-----------|
| service name           |           |
| image 또는 build context |           |
| host port              |           |
| runtime env 이름         |           |
| bind mount 필요 여부       |           |
| named volume 필요 여부     |           |
| cleanup 위험             |           |

이 표는 Day 5 통합 실습 README의 초안이 된다.